Solana Foundation: subscriptions

Security Review

Cantina Managed review by:
R0bert, Lead Security Researcher
Sujith S, Lead Security Researcher

July 30, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

The Solana Foundation is a non-profit foundation based in Zug, Switzerland, dedicated to the decentralization, adoption, and security of the Solana ecosystem.

From Jul 24th to Jul 25th the Cantina team conducted a review of subscriptions on commit hash d6b3a5dc. The team identified a total of **9** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	0	1
Medium Risk	5	2	3
Low Risk	2	1	1
Gas Optimizations	0	0	0
Informational	1	1	0
Total	**9**	**4**	**5**

The Cantina Managed team reviewed Solana Foundation's subscriptions holistically on commit hash debb4f75 and concluded that all findings were addressed and no new vulnerabilities were identified.

2.1 Scope

The security review had the following components in scope for subscriptions on commit hash d6b3a5dc:

```
program/src
├── constants.rs
├── instructions
│   ├── create_plan.rs
│   └── helpers
│       ├── plan.rs
│       ├── system.rs
│       └── transfer_utils.rs
└── state
    └── header.rs
```

3 Findings

3.1 High Risk

3.1.1 Old signed creates restore revoked permissions

Severity: High Risk.

Context: delegation.rs#L50-L98.

Description: The program uses two layers to authorize token pulls. For each `(user, mint)` pair, `initialize_subscription_authority` creates a deterministic `SubscriptionAuthority` PDA and approves it as the delegate of the user's token account for `u64::MAX`. A separate fixed delegation, recurring delegation or subscription PDA then defines who may use that authority and under which amount, period, expiry and plan restrictions.

Revoking an individual permission closes its delegation or subscription PDA. `RevokeSubscriptionAuthority` acts as the broader kill switch by revoking the token-level approval and closing the shared authority PDA. However, these accounts use deterministic addresses and their creation handlers only reject creation while the target PDA currently contains data:

```
if accounts.delegation_account.data_len() > 0 {
    return Err(SubscriptionsError::DelegationAlreadyExists.into());
}
```

Once a permission account is closed, the same address is empty again. The program keeps no tombstone, lifecycle counter, consumed authorization nonce or other persistent state showing that an earlier signed creation is no longer valid. An old `CreateFixedDelegation`, `CreateRecurringDelegation` or `Subscribe` transaction that was signed but never executed can therefore be submitted after the user has created and revoked a later permission at the same PDA. The existence check passes and the old permission is created using the terms contained in the retained signature.

This is not a replay of a transaction that already succeeded. It requires a separate signed action that has never executed, such as a durable-nonce transaction, an independently signed retry, a pending multisig proposal or a smart-wallet action. The signed terms must still be valid: for example, an expired delegation will fail and a subscription will fail if its Plan is closed, expired, sunset or no longer matches the signed terms.

Each `SubscriptionAuthority` stores an `init_id` equal to the slot in which it was created and every delegation or subscription stores a copy of that value. Normally, a creation instruction includes the exact `init_id` the user expects, which binds the new permission to the existing authority generation.

`UNKNOWN_INIT_ID` exists to support one-transaction onboarding. A client may want to place `InitSubscriptionAuthority` and a delegation creation or `Subscribe` in the same transaction, but the transaction's landing slot—and therefore the new authority's `init_id`—is not known when the user signs. When the creation instruction uses the `UNKNOWN_INIT_ID` sentinel, defined as `i64::MIN`, the program instead accepts the authority when its stored `init_id` equals the current slot. This lets an authority initialized earlier in the same transaction be used immediately, but the check is tied only to the slot and not to that particular transaction or to the time when the user approved it.

The strongest exploitation sequence is:

1. The user signs a transaction containing `InitSubscriptionAuthority` followed by `CreateFixedDelegation`, `CreateRecurringDelegation` or `Subscribe` using `UNKNOWN_INIT_ID`.
2. The transaction remains unexecuted but stays valid, for example through a durable nonce or a pending smart-wallet proposal.
3. The user later calls `RevokeSubscriptionAuthority`, which revokes the token delegate and closes the authority PDA.
4. The retained transaction is submitted.

5. `InitSubscriptionAuthority` recreates the authority PDA with `init_id` set to the new execution slot and grants it `u64::MAX` approval again.
6. The following creation accepts `UNKNOWN_INIT_ID` because the recreated authority's `init_id` equals that same current slot.
7. A new delegation or subscription is created, restoring a usable token-pull permission without any new signature from the user.

A restored fixed delegation may have no expiry, while recurring delegations and subscriptions may continue authorizing periodic pulls. An entity holding one of these retained signed actions can therefore restore spending rights after the user took an action intended to terminate them and debit the user's token account within the recreated permission's limits.

Recommendation: Store consent state that survives the closure of both the `SubscriptionAuthority` and individual permission PDAs. A dedicated PDA for each `(user, mint)` pair can contain, at minimum:

```
pub struct ConsentState {
    pub authority_epoch: u64,
    pub action_nonce: u64,
    pub approval_enabled: bool,
}
```

All authority-changing instructions should be bound to that persistent state:

- `InitSubscriptionAuthority`, `CreateFixedDelegation`, `CreateRecurringDelegation` and `Subscribe` should include the expected authority epoch, an authorization nonce and a short `valid_until_slot` or `valid_until_ts`.
- The program should reject an action when its epoch or nonce is stale or its deadline has passed.
- Successful create, revoke, initialization and reauthorization operations should advance or consume the relevant nonce so an older signed action cannot become valid again after a later lifecycle change.
- `RevokeSubscriptionAuthority` should increment `authority_epoch` and mark approval as disabled before revoking the token delegate and closing the authority PDA. Because the persistent consent PDA remains open, every action signed under the previous epoch will remain permanently invalid.
- Revoking an individual delegation or subscription must also advance a persistent permission/action generation or leave a tombstone. Merely closing the permission PDA preserves the current recreate-after-close behavior.

Remove `UNKNOWN_INIT_ID` where practical. If atomic onboarding must remain, require the Instructions sysvar to prove that the matching `InitSubscriptionAuthority` instruction appeared earlier in the same transaction for the same user, mint, token account and authority PDA. This closes the cross-transaction same-slot ambiguity, but it is not sufficient by itself: the retained malicious bundle already contains its own initialization instruction, so the persistent epoch, nonce and deadline checks are still required.

The fix must also disable the existing unbound instruction formats after migration. Adding V2 instructions while continuing to accept legacy initialization and creation instructions would leave previously signed transactions executable through the old path.

Solana Foundation: Acknowledged. Updated the doc explaining the behavior in 7ffef203.

Cantina Managed: Acknowledged.

3.2 Medium Risk

3.2.1 Stale `CancelSubscriptionNow` approvals can cancel a newly recreated subscription

Severity: Medium Risk.

Context: cancel_subscription.rs#L24-L53.

Description: `CancelSubscriptionNow` validates the subscriber, plan owner, plan address, and subscription address, but it does not bind the approval to a specific subscription incarnation.

A subscription PDA is derived only from the plan and subscriber addresses. After a subscription is cancelled and closed, the subscriber can create a new subscription at the same PDA with the same participants. Consequently, a previously signed but unsubmitted `CancelSubscriptionNow` transaction remains valid against the newly created subscription.

For example, a malicious subscriber can obtain the merchant's signature on a durable-nonce cancellation transaction for an existing subscription, withhold the transaction, close and recreate the subscription, and then submit the old transaction. All account checks still pass, causing the new subscription to expire immediately without fresh merchant approval.

This breaks the intended two-party authorization model and may allow a subscriber to prevent payment collection after receiving service.

Recommendation: Add an immutable subscription incarnation identifier, such as a user-chosen random nonce, to `SubscriptionDelegation`. Require `CancelSubscriptionNow` to include the expected identifier and reject the instruction when it does not match the live subscription.

Do not rely solely on a slot or timestamp because subscriptions may be recreated within the same slot. Consider also adding a `valid_until_slot` field to limit how long a signed cancellation transaction remains executable.

Add a regression test that signs a durable-nonce cancellation transaction, closes and recreates the subscription, and confirms that submitting the stale transaction is rejected.

Solana Foundation: We added the `expected_current_period_start_ts` in commit d4b29e8 to cancel now. Acknowledged and added a small comment to explain the same-second recreation problem in PR 222.

Cantina Managed: The team mitigated stale approvals by binding cancellation to `current_period_start_ts`, preventing replay across differing timestamps. Same-second recreation remains replayable; the team acknowledged and documented this residual risk in PR-222.

3.2.2 Stale `update_plan` approvals can restore removed pullers

Severity: Medium Risk.

Context: update_plan.rs#L109-L160.

Description: `UpdatePlanData` contains only the replacement values for `status`, `end_ts`, `pullers` and `metadata_uri`. Neither the instruction nor the `Plan` account contains a revision that binds the owner's signature to the plan state they inspected when signing.

For an Active plan, `update_plan` verifies the owner signature and the current `end_ts` constraints, then replaces every mutable field supplied by the transaction:

```
plan.status = data.status;
plan.data.end_ts = data.end_ts;
plan.data.pullers = data.pullers;
plan.data.metadata_uri = data.metadata_uri;
```

A relayer can therefore retain a distinct, fully signed update transaction that has never executed and submit it after a later owner update. For example:

1. Puller `A` is authorized on an Active plan.
2. The owner signs `U_stale`, an update that still contains `A`, but the relayer does not submit it.
3. The owner later executes `U_remove`, which removes `A` while leaving the plan Active.
4. The relayer submits `U_stale` while its transaction lifetime remains valid and its signed `end_ts` is still compatible with the live plan.

5. Because no plan revision or expected puller set is checked, `U_stale` succeeds and writes `A` back into `plan.data.pullers`.

This is not a replay of a transaction that already succeeded. The attack requires a separate signed transaction that has not previously executed, such as an older relayed update or one of multiple signed retries. A recent-blockhash transaction provides a short reordering window, while a durable-nonce transaction can preserve the signed update until the nonce is advanced.

Once the stale update lands, `Plan::can_pull` immediately treats `A` as authorized. `transfer_subscription` consults that live puller array before processing a collection, so `A` can again pull from every otherwise-valid subscription under the plan. The destination whitelist, plan expiry, subscription cancellation, SubscriptionAuthority checks and per-period allowance continue to apply. With an open destination list, the restored puller can direct funds to an account it controls; with a destination whitelist, it can still initiate charges to an allowed recipient after the owner intended to revoke that authority.

The current Sunset path is not vulnerable to this exact sequence. When a plan is already Sunset, `assert_sunset_puller_reduction` requires the new puller set to be a subset of the live set and therefore rejects re-adding an address that has already been removed. The stale-update sequence applies while the plan remains Active. It can also fail when the plan has expired or when a later update shortened `end_ts` such that the stale transaction would extend it.

The same missing lifecycle binding can affect a recreated plan at the same `(owner, plan_id)` PDA. A plan may be deleted after expiry and recreated at the same address with a new `terms.created_at`; however, `update_plan` does not include or validate that lifecycle identifier. Subject to the live `end_ts` checks, an unexecuted update signed for the previous plan can therefore modify the recreated Active plan as well.

The impact is restoration of a puller that the owner intended to revoke, enabling unauthorized recurring collections across multiple subscribers. Exploitation requires possession of a still-valid, unexecuted owner-signed update and a compatible Active plan state, making Medium severity appropriate.

Recommendation: Bind each plan update to both the current plan lifecycle and a monotonic state revision.

Add a `revision` field to the plan's mutable control state and introduce a replacement instruction such as `UpdatePlanV2` containing at least:

```
pub struct UpdatePlanV2Data {
    pub expected_created_at: i64,
    pub expected_revision: u64,
    pub valid_until_slot: u64,
    pub status: u8,
    pub end_ts: i64,
    pub pullers: [Address; 4],
    pub metadata_uri: [u8; 128],
}
```

Reject the update unless:

- `expected_created_at` matches the live plan's lifecycle identifier;
- `expected_revision` matches the live revision; and.
- The transaction lands no later than `valid_until_slot`.

After all validation succeeds, apply the update and increment the revision. This makes any previously signed update fail after a newer update executes, while `expected_created_at` prevents approvals from a deleted plan lifecycle from applying to a recreated plan at the same PDA.

The current `Plan` account has no version field or spare revision field, so the change requires an explicit account migration/reallocation or a separate versioned plan-control PDA. Use a new instruction discriminator and disable the legacy unbound `UpdatePlan` path after migration; otherwise old signed updates remain usable through the legacy instruction.

For emergency puller revocation, a dedicated remove-only instruction can provide a monotonic operation that cannot itself re-add an address. It should still update the same revision and the legacy full-replacement path must not remain available as a bypass.

Solana Foundation: Fixed in 653dec58.

Cantina Managed: Verified fix. `update_plan` now validates the signed expected lifecycle and mutable plan state, so removing a puller or recreating the plan invalidates older approvals.

3.2.3 Stale resume approvals can undo a later cancellation

Severity: Medium Risk.

Context: resume_subscription.rs#L27-L84.

Description: `ResumeSubscription` has no instruction payload that identifies the cancellation being resumed. It verifies only that the live subscription belongs to the signer, still matches the Plan and authority and currently has a future `expires_at_ts`. It then clears `expires_at_ts` to zero.

Consequently, an unexecuted resume transaction remains valid after later cancellation-state changes. For example, a subscriber can cancel, sign a resume transaction, resume through a different transaction and cancel again. A relayer that retained the first resume can submit it during the second cancellation. The current checks still pass, so the old approval clears the subscriber's newer cancellation.

This is not replay of a transaction that already executed. The attacker needs a distinct signed transaction that has not yet landed, such as a durable-nonce transaction or a pending smart-wallet action.

Once the cancellation is cleared, the merchant or an authorized puller can collect the remaining current-period allowance and future periods until the subscriber notices and disables the subscription again. A LiteSVM test reproduced the sequence with a durable-nonce resume and confirmed that the second cancellation was cleared.

Recommendation: Add a monotonic cancellation revision to `SubscriptionDelegation`. Every cancellation and resume instruction should include the expected revision, reject a mismatch and increment the revision after success.

The revision must not reset when the subscription PDA is closed and recreated. Keep the lifecycle counter in persistent state or combine it with a non-reusable subscription incarnation identifier. Add a short validity deadline to limit how long an unexecuted control transaction can remain usable.

Solana Foundation: Acknowledged and documented here: 67ad7173.

Cantina Managed: Acknowledged.

3.2.4 Stale authority-control approvals can restore or destroy later authority state

Severity: Medium Risk.

Context: close_subscription_authority.rs#L47-L50, initialize_subscription_authority.rs#L80-L131, revoke_subscription_authority.rs#L50-L84.

Description: `InitSubscriptionAuthority` does not bind the user's signature to an authority generation, action nonce, deadline or expected token-account delegate. When the authority PDA already exists, the instruction preserves its `init_id` and performs another token `Approve` CPI for `u64::MAX`.

Consequently, a relayer can retain an unexecuted initialization transaction while the authority and its grants are live. If the user later revokes the delegate directly on the token account but leaves the authority PDA open, submitting the older initialization reinstalls the same authority as delegate. Existing delegations and subscriptions become collectible again without a fresh user decision.

`CloseSubscriptionAuthority` contains no instruction data that binds the user's signature to the authority generation they intended to close. The helper verifies the signer against the live account's stored user and canonical PDA, but it does not compare a signed expected `init_id`.

`RevokeSubscriptionAuthority` has the same issue. It derives the canonical authority from the current user and mint, revokes that authority from the current ATA when present and closes the current authority account without checking which generation the user observed when signing.

A relayer can retain an unexecuted close or revoke transaction for generation `g1`. After `g1` is closed and the user deliberately creates generation `g2` at the same PDA, the relayer can submit the old transaction. It passes the live account checks and destroys `g2`; the revoke variant also removes the current token delegate.

The stale initialization can restore token-spending authority that the user disabled. The stale close and revoke actions can instead disable newly created delegations and subscriptions. LiteSVM tests reproduced both directions with durable-nonce transactions.

Recommendation: Introduce versioned initialization, close and revoke instructions that include an expected authority epoch, an action nonce and a short validity deadline. Reject each instruction unless these values match persistent consent state for the current `(user, mint)` lifecycle.

Replace the current idempotent approval refresh with a separate, explicit reauthorization instruction. Require the current epoch, a fresh nonce, a short deadline and an explicit expectation about the token account's current delegate. This prevents an initialization signed while the authority was still approved from restoring it after a direct token revocation.

Advance the action nonce after every protocol authority-control operation. Disable the legacy unbound instructions after migration, since leaving them callable would allow old signatures to bypass the new checks.

Solana Foundation: Acknowledged and documented here: 689cbbfd.

Cantina Managed: Acknowledged.

3.2.5 Subscriptions can be charged a full period at `Plan` expiry

Severity: Medium Risk.

Context: subscribe.rs#L89-L107.

Description: `Subscribe` considers a finite Plan active while `current_ts` is equal to `end_ts` and the terms signed by the subscriber do not include the Plan end timestamp. The function then starts the first billing period at the current timestamp without checking that one complete period remains before the Plan ends.

`TransferSubscription` uses the same inclusive Plan-end boundary. Therefore a merchant can submit an unexecuted subscription approval at `end_ts` and immediately collect the full per-period amount. The recorded first period starts and ends at the same timestamp. Submitting shortly before `end_ts` produces the same full allowance for a truncated first period.

This becomes more surprising when the Plan owner installs or shortens `end_ts` after the subscriber signed, since `SubscribeData` does not bind consent to that mutable field. The subscriber approved an amount and period length but can be charged the full amount for substantially less than that period.

Recommendation: Include `expected_end_ts` in `SubscribeData` and reject the instruction when it differs from the live Plan. Also require a full first period to remain:

```
current_ts
    .checked_add(period_length_s)?
    <= plan.data.end_ts
```

Skip this comparison only for perpetual Plans. Add a short subscription-approval deadline so a retained transaction cannot be delayed indefinitely even when the Plan fields remain unchanged.

Solana Foundation: Acknowledged and documented here: 37986eb.

Cantina Managed: Acknowledged.

3.3 Low Risk

3.3.1 Optional `CreatePlan` payer is missing from the IDL and generated clients

Severity: Low Risk.

Context: plan.rs#L23.

Description: The `CreatePlan` on-chain handler accepts an optional sixth account that acts as the rent payer. When supplied, this account must be a writable signer and funds the creation of the plan PDA.

However, the Codama instruction definition declares only the original five accounts and does not include the optional payer. Consequently, the generated IDL and clients do not accurately describe the on-chain interface.

The handwritten TypeScript overlay works around this by manually appending the payer, but other consumers of the IDL do not receive this behavior. In particular, the raw TypeScript builder cannot provide a named payer, while Rust callers must pass it as an untyped remaining account.

This inconsistency can cause sponsored plan creation to fail or be integrated incorrectly by applications relying on the published IDL and generated clients.

Recommendation: Add the optional payer to the Codama account definition for `CreatePlan` as a writable signer using the omitted-account strategy. Regenerate the IDL and TypeScript/Rust clients afterward.

Update the TypeScript overlay to pass the payer through the generated builder instead of manually appending it, and add client tests verifying the payer's position, signer status, and writable status.

Solana Foundation: Fixed in 11173179.

Cantina Managed: Verified fix.

3.3.2 Automatic rent sponsorship can lock relayer funds indefinitely

Severity: Low Risk.

Context: close_subscription_authority.rs#L6-L29, close_subscription_authority.rs#L47-L50, authority.rs#L10-L40, initialize_subscription_authority.rs#L18-L52, initialize_subscription_authority.rs#L82-L105.

Description: When `client.payer` differs from `client.identity`, the TypeScript plugin automatically appends the payer as the rent sponsor for authority initialization, delegation creation, subscription creation and Plan creation.

This silently changes the fee payer into a long-lived capital provider. Several sponsored accounts cannot be reclaimed by the sponsor while they remain healthy:

- A sponsored `SubscriptionAuthority` may only be closed by the user;
- A non-expiring recurring delegation can remain open forever;
- A non-expiring fixed delegation with remaining allowance can remain open forever;
- An active perpetual subscription can retain rent indefinitely;
- Abandonment recovery becomes available only after the user closes or rotates the authority;
- Sponsored Plan rent is paid to the merchant on deletion rather than returned to the sponsor.

A malicious user can create many sponsored accounts and leave them open, while an inactive user can create the same result unintentionally. The plugin does not require an explicit acknowledgment of this liability.

Recommendation: Do not infer rent sponsorship from the transaction fee payer. Require an explicit `payer` or `sponsorRent` option for each instruction.

Sponsored state should include enforceable recovery terms, such as a finite sponsor deadline, mandatory expiry, maximum account lifetime or an escrowed refund design. Relayers should also enforce per-user and aggregate rent quotas and reject non-expiring sponsored permissions unless the protocol provides a unilateral recovery path.

Solana Foundation: Acknowledged. Added a comment on TS Readme here (was only in code): 0ac23b9f7, but this is a feature we want to keep for usability.

Cantina Managed: Acknowledged.

3.4 Informational

3.4.1 IDL generation failures do not stop program and client releases

Severity: Informational.

Context: build.rs#L12-L14.

Description: `main` catches every error returned by `generate_idl` and emits a Cargo warning instead of terminating the build. The build script therefore exits successfully even when Codama cannot load the program sources, cannot generate valid JSON or cannot write `idl/subscriptions.json`.

The repository's generation commands trust that successful exit status. `just generate-idl` prints that the IDL was generated after a failure, while `just check-generated` can rebuild clients from the existing stale IDL and report that all generated files are current.

The release workflow runs `just generate-idl` before building the verified program, but it does not compare the resulting IDL or generated clients with the current program. The TypeScript and Rust publication workflows have the same weakness: `cargo build` may log the generation failure, after which client generation reads the existing IDL and packaging continues.

A reversible test made `idl/subscriptions.json` unwritable. Both `just generate-idl` and `just check-generated` logged `Failed to generate IDL: Permission denied` and exited with status zero. The TypeScript publication commands then built `dist` and produced the `@solana/subscriptions` package tarball from the unchanged IDL.

Consequently, a Rust or Codama annotation change that still compiles but prevents IDL generation can be released with stale generated interfaces. Applications may construct instructions with obsolete account layouts or data formats and indexers may decode new program data using old schemas.

Recommendation: Propagate `generate_idl` errors from the build script so Cargo exits with a nonzero status:

```rust
fn main() -> Result<(), Box<dyn std::error::Error>> {
    println!("cargo:rerun-if-changed=src/");
    println!("cargo:rerun-if-env-changed=GENERATE_IDL");
    generate_idl()
}
```

Require `just check-generated` in every program release and client publication job before any program buffer or package artifact is created. The check should fail if generation fails and should compare the freshly generated IDL and both generated clients against their committed versions.

Solana Foundation: Fixed in 1b6e98e.

Cantina Managed: Verified fix.